Developmentof Python Code Compable withMulpleAI Tools

Name: HARINI S K

Reg.no: 212225230092

Course:Prompt

Engineering Slot:T2-H16

Introduc on

Artificial Intelligence (AI) tools such as ChatGPT, Goog Claude AI are widely used for generating Python programs, so problems, and interacting with APIs. Instead of directly writing developers can create effective prompts that guide AI systems and reusable Python code.

This topic focuses on developing Python code compatible with using prompt engineering techniques.

Objectives

The main objectives are:

1. To understand prompt-based Python code generation.

2. To create prompts for interacting with APIs.
3. To compare outputs generated by multiple AI tools.
4. To evaluate prompt effectiveness.
5. To refine prompts for better results.

Importance of AI-Assisted Coding

AI-assisted coding helps:

- Reduce development time
- Improve coding efficiency
- Generate documentation automatically
- Debug errors quickly
- Learn programming concepts interactively

What is an API?

- API stands for Application Programming Interface. APIs all
communicate with each other.

Example

- A weather application uses a weather API to fetch tempera
- Python commonly uses:
- REST APIs
- Web APIs

- AI APIs
- Database APIs

- Role of Prompt Engineering in Coding
- Prompt engineering helps developers:
- Generate structured Python programs
- Control output quality
- Reduce syntax errors
- Improve code readability

Understanding Multi-AI Tool Compatibility

What is Multi-AI Compatibility?

Multi-AI compatibility means creating prompts and Python work effectively across different AI platforms.

Popular AI Tools

- ChatGPT
- Google Gemini
- Claude AI
- Microsoft Copilot

Each AI tool may generate slightly different responses for t

Why Compare AI Outputs?

Comparing outputs helps:

- Identify the best coding style
- Detect errors or inefficiencies
- Improve prompt quality
- Understand strengths of each AI system

Common Differences Between AI Tools

Feature	ChatGPT	Gemini	Claude
Code Explanation	Detailed	Moderate	Very Detailed
Creativity	High	Medium	High
API Integration Support	Strong	Strong	Moderate
Error Handling	Good	Moderate	Good

Benefits of Multi-AI Testing

- Better accuracy
- Improved reliability
- More coding alternatives
- Enhanced learning experience

Prompt Design for Python Code Generation

What is a Prompt?

A prompt is an instruction given to an AI tool.

Example Prompt

"Generate a Python program to fetch weather data using a

Characteristics of a Good Prompt

A good prompt should be:

Clear

Specific

- Context-aware
- Goal-oriented

Basic Prompt Structure

1. Task Description

Describe what the AI should do.

2. Programming Language

Specify Python clearly.

3. Requirements

Mention libraries, APIs, and output format.

4. Constraints
Mention limitations if needed.

Example of a Weak Prompt
“Write API code.”

Problems

- No API specified
- No programming language specified
- Unclear output expectations

Example of a Strong Prompt
“Generate a Python program using the requests library to f
from a REST API and display temperature, humidity, and c

Advantages

- Clear instructions
- Specific requirements
- Better output quality

Prompting Stages in AI-Based Python Developm

Stage 1 – Prompt for API Interaction

Example Prompt

“Generate Python code using the requests library to intera
API and display JSON responses.”

Expected Output

- API request code
- JSON parsing
- Exception handling

Stage 2 – Prompt for Data Comparison

Example Prompt

“Compare outputs from two APIs and highlight differences in re
format.”

Expected Output

- Comparison table
- Performance analysis
- Error comparison

Stage 3 – Prompt for Insights

Example Prompt

“Analyze API responses and suggest meaningful insights f
data.”

Expected Output

- Trend analysis
- Suggestions
- Recommendations

Importance of Multi-Stage Prompting

- Breaking tasks into stages:
- Improves accuracy
- Reduces confusion

Produces organized outputs

Python Code Generation for API Interaction Python

- Libraries Used
- Python commonly uses:

requests

json

- pandas
- matplotlib

Example Python Workflow

Step 1 – Send API Request
Python sends a request to the API server.
Step 2 – Receive JSON Data
The API returns data in JSON format.
Step 3 – Process Data
Python extracts useful information.
Step 4 – Display Results
Results are printed or visualized.

Example Prompt

“Generate Python code to connect to a public API and disp
information in table format.”

Expected AI-Generated Features

- API connection
- Data extraction

- Error handling
- Formatted output

Benefits of AI-Generated Python Code

- Faster development
- Reduced manual coding
- Easy debugging
- Learning support for beginners

Comparative Analysis of AI Tool Outputs

Purpose of Comparative Analysis

Comparative analysis helps evaluate:

- Accuracy
- Readability
- Performance
- Error handling

Example Scenario

Prompt Given to AI Tools

“Generate Python code for fetching cryptocurrency prices using an API.”

Criteria	ChatGPT	Gemini	Claude
Code Readability	High	Medium	High
Error Handling	Strong	Moderate	Strong
Comments in Code	Detailed	Limited	Detailed
API Explanation	Excellent	Good	Excellent

Observations

- ChatGPT generated beginner-friendly code.
- Gemini generated concise code.
- Claude provided detailed explanations.

Conclusion from Comparison

Different AI tools are useful for different coding requirements

Screenshots and Documentation Requirements

Importance of Screenshots

Screenshots provide proof of:

- Prompt usage
- AI-generated outputs
- Comparative testing

Required Screenshots

Learners should include:

1. Prompt entered into AI tool
2. Generated Python code
3. Output/result screen
4. Comparison between AI tools

Documentation Format

Section 1 – Prompt Used
Include exact prompts.
Section 2 – AI Responses
Include generated code and explanation.
Section 3 – Comparative Analysis
Compare responses from at least two AI tools.
Section 4 – Reflection Note
Explain prompt refinement and learning experience.

Recommended Tools for Documentation

- Microsoft Word
- Google Docs
- Canva

Reflection Note on Prompt Effectiveness

What is a Reflection Note?

A reflection note explains:

- What worked well
- What challenges occurred
- How prompts were improved

Key Points to Include

1. Initial Prompt Quality

Describe whether the first prompt produced good results.

2. Problems Faced

Examples:

Incomplete code

Missing error handling

Incorrect API usage

- 3. Prompt Refinement

Explain how prompts were modified.

Example

Initial Prompt:

“Generate API code.”

Refined Prompt:

“Generate Python code using requests library to fetch live w
proper exception handling.”

Benefits of Reflection

- Improves prompt-writing skills
- Enhances AI understanding
- Helps future coding tasks

Challenges and Future Scope

Challenges in AI-Based Code Generation

1. Inconsistent Outputs

Different AI tools may produce different code.

2. API Authentication Issues

Some APIs require tokens or API keys.

3. Syntax Errors

Generated code may sometimes contain mistakes.

4. Dependency Problems

Libraries may not be installed properly.

Solutions

- Use clear prompts
- Mention required libraries
- Specify output format
- Test generated code carefully

Future Scope

AI-Assisted Development

Future programming may heavily depend on AI-generated

Smart Prompt Systems

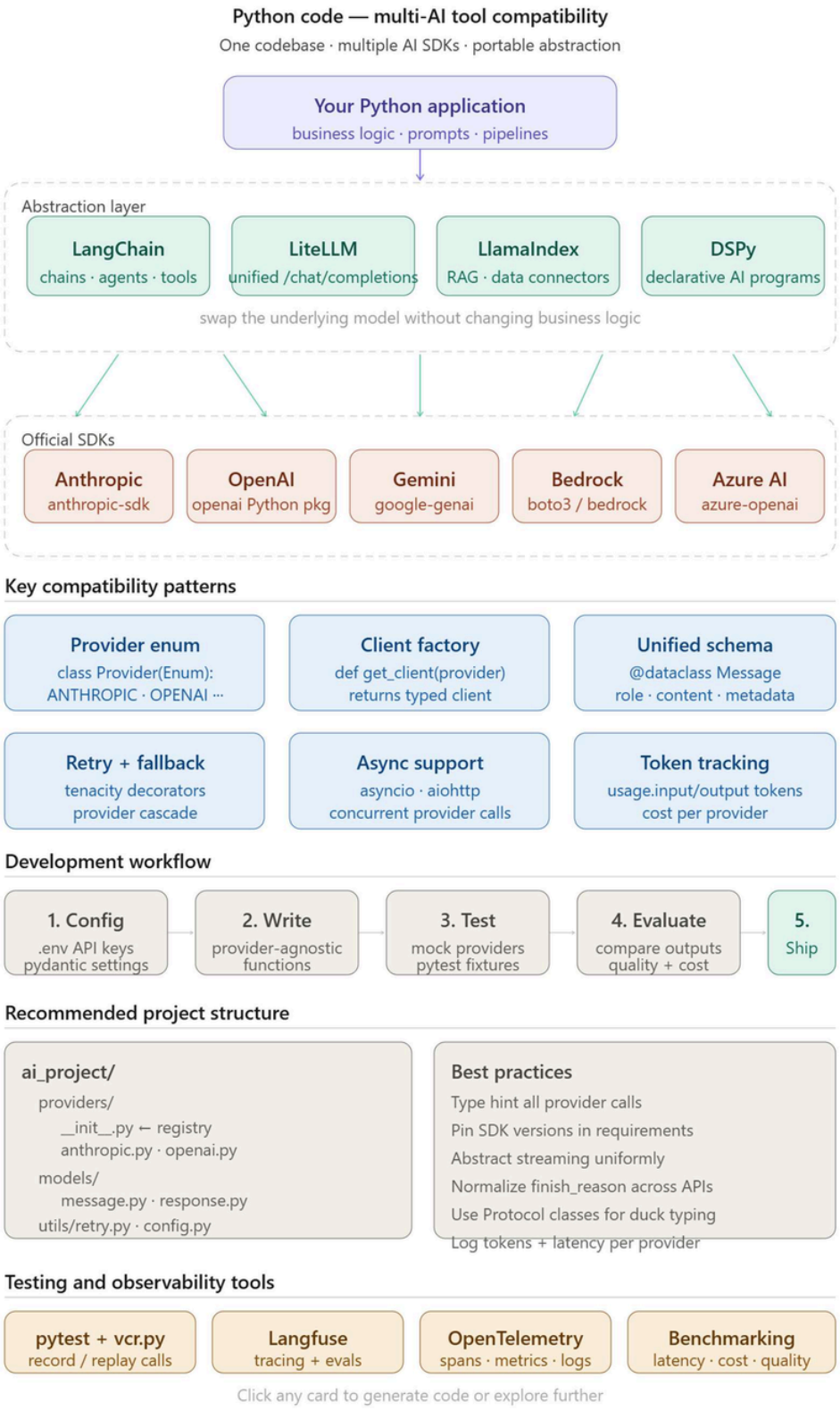
AI systems may automatically optimize prompts.

Advanced API Automation

AI tools may create fully automated API workflows.

Collaborative AI Coding

Multiple AI systems may work together on software develo



Conclusion

The development of Python code compatible with multiple
important application of prompt engineering. Instead of ma
programs, developers can design prompts that guide AI sy
efficient Python code for API interaction and data analysis.

Final Findings

Different AI tools provide different advantages:

- ChatGPT provides detailed explanations and beginner-frie
- Google Gemini generates concise and fast responses.
- Claude AI offers detailed reasoning and documentation.